🚙 Proyecto: Sistema de Estacionamiento Inteligente (Inalámbrico)

1. Descripción General de la Idea

Evolucionar un sistema técnico de control de estacionamiento hacia un producto comercializable, modular y robusto para cocheras de edificios o playas privadas.

- **Fase 1 (MVP actual):** Enfoque exclusivo en detección mediante sensores y visualización centralizada, eliminando barreras mecánicas para reducir costos y agilizar la instalación.
- **Conectividad IoT sin cables:** Uso de mini ESP32 distribuidos en cada cajón, comunicados por Wi-Fi con una central, evitando tirar metros de cables molestos.

2. Componentes Necesarios (Para una prueba de 3 o 4 lugares)

📍 **Para los Nodos de Sensores (por cada punto de estacionamiento):**

- **Sensores Infrarrojos:** Módulos **TCRT5000** (con potenciómetro de sensibilidad y salida digital).
- **Mini ESP32:** Modelo **NodeMCU ESP32-C3 SuperMini** (diminutos, con Wi-Fi y Bluetooth, puerto USB-C).
- **Alimentación:** Fuentes de alimentación de 5V (cargadores de celular viejos con cable USB-C) para enchufar cada nodo a la pared de forma permanente.

🧠 **Para la Central (El Cerebro Receptor):**

- **1 Placa ESP32 estándar** (por ejemplo, NodeMCU ESP32 WROOM) para recibir los avisos por Wi-Fi y procesar el conteo general.
- **1 Fuente de 5V** para mantener la central encendida.

3. Conexión Física de los Nodos Remotos

Para armar cada uno de los nodos remotos con el Mini ESP32-C3 y el sensor TCRT5000, solo necesitas conectar **4 pines** (puedes usar cables Dupont hembra-macho):

- **Pines del Sensor TCRT5000 ➡️ Pines del Mini ESP32-C3:**
    1. **VCC del sensor ➡️ 3.3V** (o 5V según la versión del módulo) del Mini ESP32.
    2. **GND del sensor ➡️ GND** del Mini ESP32.
    3. **OUT (Salida digital) del sensor ➡️ Pin GPIO 4** (o cualquier pin digital libre) del Mini ESP32. *(Este pin es el que lee el código).*

4. Resumen de Funcionamiento en el Banco de Pruebas

1. **Alimentación:** Conectas un cargador de celular común a la pared y un cable USB-C al Mini ESP32-C3 que tiene el sensor TCRT5000.
2. **El "Cerebro":** Enchufas el ESP32 grande en tu escritorio (la central) y le cargas el código del servidor.
3. **La magia del Wi-Fi:** Cuando acerques un objeto (o un auto de juguete) al sensor TCRT5000, este cambiará su estado, el Mini ESP32-C3 oirá el cambio, enviará un aviso invisible por el Wi-Fi de tu casa, y la central lo imprimirá al instante en el monitor de tu computadora.

5. Programación (Códigos Listos para Copiar y Pegar)

Código 1: Para cada Mini ESP32 (Nodo Sensor Remoto)

Lee el sensor infrarrojo y, solo cuando detecta un cambio de estado, se conecta por Wi-Fi para avisar al ESP32 principal.

```
#include <WiFi.h>

#include <HTTPClient.h>


// Datos de tu red Wi-Fi

const char* ssid     = "TU_RED_WIFI";

const char* password = "TU_PASSWORD";

// Dirección IP del ESP32 central (ejemplo: 192.168.1.100)

const char* servidorCentral = "http://192.168.1.100/actualizar";

const int pinSensor = 4; // Pin digital del mini ESP32 conectado al OUT del sensor infrarrojo

const int idPlaza = 1;   // Cambiar este número según la plaza (1, 2, 3, etc.)

int estadoAnterior = -1; // Variable para registrar el estado previo y detectar cambios


void setup() {

  Serial.begin(115200);

  pinMode(pinSensor, INPUT);
```

```
  WiFi.begin(ssid, password);
  Serial.print("Conectando nodo a Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nNodo conectado al Wi-Fi con éxito!");
}

void loop() {
  int lecturaActual = digitalRead(pinSensor);

  // Lógica del sensor IR (TCRT5000):
  // LOW (0) = Objeto detectado (Ocupado), HIGH (1) = Libre.
  int estadoActual = (lecturaActual == LOW) ? 1 : 0;

  // AHORRO: Solo actúa y transmite si el estado CAMBIÓ respecto al ciclo anterior
  if (estadoActual != estadoAnterior) {
    estadoAnterior = estadoActual;

    Serial.print("¡Cambio en Plaza ");
    Serial.print(idPlaza);
    Serial.print("! Estado: ");
    Serial.println(estadoActual == 1 ? "OCUPADO" : "LIBRE");

    if (WiFi.status() == WL_CONNECTED) {
      HTTPClient http;
```

```
    String url = String(servidorCentral) + "?plaza=" + idPlaza + "&estado=" + estadoActual;

    http.begin(url);
    int codigoRespuesta = http.GET();
     if (codigoRespuesta > 0) {
      Serial.println("Aviso enviado correctamente a la central.");
    } else {
      Serial.println("Error al enviar el aviso.");
    }
    http.end();
   }
  }
 delay(500); // Pequeña pausa para evitar rebotes eléctricos
}
```

Código 2: Para el ESP32 Principal (El Cerebro Receptor)

Levanta un servidor web interno en la red Wi-Fi para escuchar los avisos de los mini ESP32 y calcular los lugares libres/ocupados en tiempo real

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid     = "TU_RED_WIFI";
const char* password = "TU_PASSWORD";

// IP fija opcional para la central (para que siempre sea la 192.168.1.100)
IPAddress local_IP(192, 168, 1, 100);
IPAddress gateway(192, 168, 1, 1);
IPAddress subnet(255, 255, 255, 0);
```

```
WebServer server(80);

const int totalLugares = 4;          // Cantidad de plazas en la prueba
int estadosPlazas[4] = {0, 0, 0, 0}; // 0 = Libre, 1 = Ocupado

void setup() {
  Serial.begin(115200);

  if (!WiFi.config(local_IP, gateway, subnet)) {
    Serial.println("Falla al configurar la IP estática");
  }

  WiFi.begin(ssid, password);
  Serial.print("Conectando central a Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nCentral conectada con éxito!");
  Serial.print("IP del Central: ");
  Serial.println(WiFi.localIP());

  server.on("/actualizar", handleActualizacion);
  server.begin();
  Serial.println("Servidor central listo y escuchando a los nodos.");
}

void loop() {
```

```
  server.handleClient();
}

void handleActualizacion() {
  if (server.hasArg("plaza") && server.hasArg("estado")) {
    int plaza = server.arg("plaza").toInt() - 1; // Ajustar índice (0 a 3)
    int estado = server.arg("estado").toInt();

    if (plaza >= 0 && plaza < totalLugares) {
      estadosPlazas[plaza] = estado;

      int ocupados = 0;
      for (int i = 0; i < totalLugares; i++) {
        ocupados += estadosPlazas[i];
      }
      int libres = totalLugares - ocupados;

      Serial.println("\n--- ACTUALIZACIÓN DE ESTACIONAMIENTO ---");
      Serial.print("Plaza "); Serial.print(plaza + 1);
      Serial.println(estado == 1 ? " -> OCUPADA" : " -> LIBRE");
      Serial.print("Lugares Libres: "); Serial.println(libres);
      Serial.print("Lugares Ocupados: "); Serial.println(ocupados);
      Serial.println("----------------------------------------");

      server.send(200, "text/plain", "OK");
    } else {
      server.send(400, "text/plain", "Plaza fuera de rango");
    }
```

```
  } else {
    server.send(400, "text/plain", "Faltan datos en la peticion");
  }
}
```